BMS INSTITUTE OF TECHNOLOGY AND MANAGEMENT

Autonomous Institute under VTU, Belagavi, Karnataka - 590 018

Yelahanka, Bengaluru, Karnataka - 560 119



Information and Network Security (BCS701) CCA Report

On

SECURE AUTHENTICATION FOR IoT DEVICES

BACHELOR OF ENGINEERING

in

INFORMATION SCIENCE AND ENGINEERING

by

NAME: _____ USN: _____

Under the Guidance of

Dr. Swetha M S

Associate Professor

DEPARTMENT OF INFORMATION SCIENCE AND ENGINEERING

Avalahalli, Yelahanka, Bengaluru, Karnataka - 560119

2025-26

Evaluation sheet

Name	USN	Problem Understanding & Requirements	Design & Implementation	Security Analysis & Testing	Innovation & Application Relevance	Documentation & Presentation	Total
		3	9	2	2	4	20

Content Page No.

1. Introduction 4

2. Abstract 4

3. Problem Statement 4

4. System Requirements 5

5. Design and Implementation 6

6. Security Analysis 7

7. System Testing 8

8. Innovation and Application relevance 8

9. Conclusion 8

10. Referencee 9

1. Introduction

Internet of Things (IoT) deployments connect sensors, controllers, gateways, and cloud services that may be unattended and resource-constrained. Each device must establish its identity before it can publish readings or accept commands. If an attacker can impersonate a device, inject messages, or reuse captured authentication data, the system may make unsafe decisions or lose trustworthy telemetry.

This project implements a compact challenge-response prototype. A device proves possession of a provisioned secret by computing a keyed message authentication code (HMAC) over a fresh server challenge, its device identity, and a one-time session identifier. The design follows the basic security goals of authenticating peers and protecting communications; deployment guidance also emphasizes lifecycle controls for IoT products [1], while HMAC is specified as a keyed message authentication construction [2].

The prototype is intentionally small enough to study. It contains a local authentication server, a simulated device, a JSON demonstration registry, and a report. The source files are in the accompanying *iot-auth-project* folder.

2. Abstract

This report presents a secure authentication prototype for IoT devices using nonce-based challenge-response authentication. The server issues a cryptographically random 256-bit challenge associated with a registered device and a unique session identifier. The simulated device computes an HMAC-SHA-256 proof using its pre-provisioned symmetric key. The server verifies the proof with a constant-time comparison and consumes the session regardless of success, limiting reuse of a captured response. Challenges expire after 60 seconds.

The prototype is implemented with Python's standard library and exposes two JSON endpoints: `/challenge` and `/authenticate`. Optional TLS can be enabled by providing a certificate and private key. The design meets the educational objectives of demonstrating identity validation, integrity, freshness, and replay resistance. The evaluation is a design-based security analysis and a documented test plan; it does not claim production readiness or formal verification. Real deployments additionally require protected per-device keys, verified TLS, authorization, revocation, rotation, monitoring, and fault-tolerant replay state.

Keywords: IoT security, device identity, HMAC, challenge-response, nonce, replay resistance.

3. Problem Statement

A large IoT network may contain devices that communicate over shared or exposed networks. A static device password or reusable MAC value can be copied from traffic and replayed. Weak identity checks may also accept unregistered devices, while inconsistent key handling can expose the whole deployment.

The problem addressed is to demonstrate a lightweight authentication flow that lets a server distinguish a registered device possessing its assigned secret from an unauthenticated claimant, while making each proof specific to one fresh session.

Objectives

- Register demonstration device identities with individual symmetric keys.
- Issue unpredictable challenges with bounded validity.

- Bind the authentication proof to the device ID, challenge, and session ID.
- Reject unknown devices, invalid proofs, expired sessions, and reused sessions.
- Document the security properties, limits, and future deployment requirements.

Scope and assumptions

The demonstration assumes an attacker may observe, modify, delay, or replay network messages, but cannot read the server's device registry or extract the device's secret key. The prototype does not implement device onboarding, authorization of individual commands, protected hardware storage, certificate issuance, or a production database. Those assumptions are explicit because authentication alone does not establish that every authenticated device is authorized for every action.

4. System Requirements

Functional requirements

ID	Requirement	Implementation
FR-1	Identify a device using a registered device ID.	JSON device registry
FR-2	Generate a fresh server challenge and session ID.	Python secrets module
FR-3	Compute and verify an HMAC proof over all session context.	HMAC-SHA-256
FR-4	Reject an unknown device or malformed request.	Input checks and 4xx responses
FR-5	Expire and consume each challenge at most once.	60-second TTL and atomic pop
FR-6	Support encrypted transport for a local demonstration.	Optional Python TLS socket

Non-functional requirements

- Use standard cryptographic primitives from the Python standard library; do not invent a MAC algorithm.
- Keep the demo runnable without third-party Python packages.
- Use a constant-time comparison for the supplied proof.
- Return clear authentication outcomes without disclosing secret material.
- Keep the sample deployment local by default and identify production security gaps.

Hardware and software

The prototype runs on a computer with Python 3.10 or newer. A real IoT target needs a network interface, a cryptographic random source, and protected persistent storage for a unique device key. The demo server binds to 127.0.0.1 by default; HTTPS can be enabled with a trusted certificate and matching client configuration.

5. Design and Implementation

Architecture

The system has three logical components: (1) an authentication server holding the registered device keys and pending challenges, (2) a device simulator holding the matching demo key, and (3) an HTTP(S) JSON interface connecting them. The server does not transmit a device key. It sends a random challenge, and the client returns only a keyed proof.

Step	Message / action	Security purpose
1	Device → server: POST /challenge {device_id}	Requests a session for a known identity
2	Server → device: random challenge + session_id + expiry	Fresh unpredictable verifier input
3	Device: HMAC(key, context || ID || session || challenge)	Proves key possession; binds context
4	Device → server: POST /authenticate {ID, session, proof}	Submits one session-specific proof
5	Server: consume session, recompute, constant-time compare	Rejects replay and avoids simple timing leaks

Authentication message

```
message = b"INS-IOT-AUTH-v1" + b"|" + device_id
          + b"|" + session_id + b"|" + challenge
proof   = HMAC-SHA-256(device_key, message)
```

The fixed domain label separates this use from other possible uses of the same key. The challenge and session identifier are generated by the server; the client does not choose either. The server stores pending state under a random session ID and removes it before proof comparison, so a failed attempt cannot reuse that challenge. An expiry timestamp bounds how long abandoned challenges remain usable.

Project structure

```
iot-auth-project/
  server.py                challenge service and verifier
  device.py                device simulator
  data/demo_devices.json   public classroom sample identities
  README.md                setup and security limitations
  report/build_report.py   report source
  report/Secure_Authentication_for_IoT_Devices_CCA_Report.pdf
```

Run the server with `python server.py`, then use another terminal to run `python device.py sensor-001`. The expected successful result is a JSON response with `authenticated: true`. The sample keys are deliberately public and must never be reused.

6. Security Analysis

Security objectives

Property	How the prototype supports it	Boundary
Device authentication	Proof is checked against the key registered for the claimed ID.	Compromised key allows impersonation of that device.
Integrity of proof	HMAC-SHA-256 detects a modified proof or message context.	Authentication proof does not encrypt telemetry.
Freshness	A random challenge is tied to a short-lived session.	In-memory state is lost on server restart.
Replay resistance	The session is consumed once before verification.	A multi-server deployment needs shared atomic state.
Transport confidentiality	Optional TLS socket support.	HTTP demo mode must remain on localhost.

Threat analysis

- Replay: an earlier proof includes an old challenge and session. Once consumed or expired, that session is no longer accepted.
- Impersonation: a claimant without the registered secret cannot feasibly create the expected HMAC, assuming a strong secret and sound implementation.
- Unknown identity: challenge issuance is refused when the device ID is absent from the registry.
- Tampering: changing the device ID, session ID, or challenge changes the authenticated message and invalidates the proof.
- Eavesdropping: the HMAC does not conceal readings or API metadata. TLS is required for confidentiality and channel protection. TLS 1.3 is designed to protect against eavesdropping, tampering, and message forgery [3].

Key management and limitations

Key secrecy is the central assumption. The checked-in JSON keys are public test fixtures, the file is unencrypted, and a copied key permits device impersonation. Before real deployment, provision a distinct high-entropy key per device through an authenticated process, store it in a secure element or equivalent protected hardware, and design revocation, replacement, and audit procedures. NIST's IoT manufacturer guidance emphasizes foundational activities across device lifecycles [1]. NIST defines replay resistance as protection from capturing and retransmitting authentication information [4].

The server uses process-local challenge memory. A restart clears active sessions, which is fail-closed for these sessions, but concurrent server instances do not share replay state. Production operation needs a durable, atomic challenge store, request rate limits, monitoring, authorization checks, secure backups, and operational key rotation. The prototype should not be represented as certified or production hardened.

7. System Testing

The following verification cases are defined for demonstration and evaluation. They describe expected behavior; no measured execution results are claimed in this report.

Test	Input / action	Expected result
ST-1	Authenticate sensor-001 with its demo key.	HTTP 200 and authenticated=true.
ST-2	Request a challenge for an unknown device ID.	HTTP 401; no challenge is issued.
ST-3	Change one character of a valid proof.	HTTP 401; session is consumed.
ST-4	Submit the same session and proof a second time.	HTTP 401 because the session no longer exists.
ST-5	Submit an expired challenge after 60 seconds.	HTTP 401; expired proof is rejected.
ST-6	Send missing fields or malformed JSON.	HTTP 400 with a generic request error.
ST-7	Enable TLS with a trusted local certificate.	Client connects only after certificate validation.

Evaluation measures

The implementation is small and deterministic in its protocol shape, uses standard library cryptography interfaces, and keeps the secret off the wire. Functional acceptance should be established by completing the scenarios above. Security acceptance additionally requires code review of key storage, deployment configuration, TLS certificate validation, session-state consistency, logging, and authorization. Performance and memory measurements are outside this classroom prototype's current scope.

8. Innovation and Application relevance

The key design choice is the use of a one-time server challenge with the device identity and session identifier included in the HMAC input. This keeps the protocol understandable while addressing the common weakness of repeatedly transmitting a static credential. Explicit session consumption also makes replay handling visible in the implementation, rather than relying on timestamps alone.

The prototype is relevant to smart home sensors, environmental monitoring, industrial telemetry, and classroom IoT testbeds where a gateway must establish device identity before accepting data. Its standard-library implementation lowers setup friction for learning and makes the message flow easy to adapt to constrained clients.

Potential extensions include mutual authentication, certificate-based identity, per-device authorization policies, secure enrollment, key rotation, revocation lists, persistent clustered replay state, and integration with MQTT over verified TLS. Any extension should preserve the same principles: unique identity, protected credentials, cryptographic freshness, authenticated transport, and a clear lifecycle plan.

9. Conclusion

This project demonstrates a practical device-authentication flow using a fresh challenge and an HMAC-SHA-256 proof. The server validates registered identities, consumes sessions once, and enforces a short expiry. The accompanying simulator provides a concrete way to inspect the protocol, while the report maps the implementation to system requirements and threat scenarios.

The prototype is suitable for coursework and local experimentation. Its most important deployment lesson is that a sound proof algorithm depends on sound key provisioning, protected storage, trusted transport, authorization, and lifecycle management. These controls must be added before adapting the demonstration for real IoT devices.

10. Referencee

[1] National Institute of Standards and Technology, *Foundational Cybersecurity Activities for IoT Product Manufacturers*, NISTIR 8259 Rev. 1, 2026. Available: https://csrc.nist.gov/pubs/ir/8259/r1/final

[2] H. Krawczyk, M. Bellare, and R. Canetti, *HMAC: Keyed-Hashing for Message Authentication*, RFC 2104, Internet Engineering Task Force, 1997. Available: https://www.rfc-editor.org/rfc/rfc2104

[3] E. Rescorla, *The Transport Layer Security (TLS) Protocol Version 1.3*, RFC 8446, Internet Engineering Task Force, 2018. Available: https://www.rfc-editor.org/rfc/rfc8446

[4] National Institute of Standards and Technology, *Digital Identity Guidelines: Authentication and Authenticator Management*, NIST SP 800-63B-4, 2025. Available: https://csrc.nist.gov/pubs/sp/800/63/B/4/final